

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL-2 COMPARATIVE ANALYSIS
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:	KAMALESH K	Reg.No.:	192572164
Department:	BE.CSE(AI)	Date:	23/082026

1. Comparison of L1, L2, and L3 Cache Levels

Introduction

Cache memory is a small and fast memory system located close to the processor. Modern processors generally use multiple cache levels to reduce the time required to obtain frequently used instructions and data. L1 is normally the smallest and fastest cache, while L2 is larger but slower, and L3 is generally the largest and slowest cache level. In multicore processors, L3 is commonly shared among several cores. Intel documentation describes L1 as the fastest cache and L3 as the larger, slower last-level cache.

Feature	L1 Cache	L2 Cache	L3 Cache
Relative size	Smallest	Medium	Largest
Relative latency	Lowest	Higher than L1	Higher than L2
Typical latency	~1–4 cycles	~4–12 cycles	~30–50 cycles in many designs
Bandwidth	Very high	Very high	High, but lower effective access speed
Location	Closest to CPU core	Close to individual/core group	Usually shared among cores

Main purpose	Immediate instruction/data access	Back up L1 misses	Reduce accesses to main memory
Cost per bit	Highest	High	Lower than L1
Impact on performance	Very high	High	High for multicore workloads

Representative architectural figures show L1 access around a few cycles, L2 around several to roughly a dozen cycles, and L3 substantially higher. For example, Intel has published examples with L1 around 4 cycles, L2 around 12 cycles, and L3 around 26–31 cycles.

L1 Cache

L1 is the first cache checked by the processor. It is normally divided into instruction and data caches. Its small capacity allows it to use very fast circuits and short physical paths. Because of its low latency and high bandwidth, a successful L1 hit allows the processor to continue execution with minimal delay.

L2 Cache

L2 is larger than L1 and acts as the next level when the required information is not found in L1. Its greater capacity improves the probability of finding recently used information, but the larger structure increases access latency. In many modern processors, L2 is private to a core or shared within a small group of cores. Current Intel processor documentation provides examples of private L2 caches for performance cores and shared L2 structures for groups of efficiency cores.

L3 Cache

L3, also called the Last-Level Cache (LLC), is normally larger than L1 and L2 and is often shared by multiple processor cores. Its purpose is to reduce the number of expensive accesses to DRAM. Because it is larger and farther from an individual execution core, its latency is higher than L1 and L2. However, its shared nature is particularly useful in multicore systems because data can be accessed by different cores.

Impact on Processor Performance

The cache hierarchy improves processor performance by exploiting **temporal locality** and **spatial locality**. If a program repeatedly uses the same data, it is likely to remain in a cache. If the processor misses in L1 but finds the data in L2, the penalty is much smaller than accessing main memory. An L3 hit is slower than an L1 or L2 hit but can still avoid a much more expensive DRAM access.

Therefore, the hierarchy provides a compromise:

L1 → fastest but smallest

L2 → larger with moderate latency

L3 → largest cache with higher latency and multicore sharing

Intel reports representative memory-system values of approximately 1 ns for L1, 4 ns for L2, and a substantially slower L3, while noting that actual values vary by hardware.

Critical Analysis

Increasing cache size does not automatically increase performance. A very large cache can increase access latency, area, and power consumption. Therefore, processor designers use multiple levels instead of making one enormous cache. The hierarchy provides high-speed access for the most frequently used information while still providing greater capacity at lower levels.

Conclusion

L1, L2, and L3 caches form an essential part of modern multicore processor architecture. L1 provides the lowest latency, L2 provides additional capacity with moderate latency, and L3 provides a large shared buffer before main memory. Effective cache utilization reduces average memory access time and improves processor throughput.

2. Comparison of SRAM and DRAM

Introduction

SRAM and DRAM are both volatile semiconductor memories, but they have different cell structures and performance characteristics. SRAM uses a flip-flop-based cell and does not require periodic refresh, whereas DRAM stores information as electrical charge and therefore requires refresh operations.

Feature	SRAM	DRAM
Full form	Static Random Access Memory	Dynamic Random Access Memory
Storage cell	Typically flip-flop based	Typically 1 transistor + capacitor
Refresh	Not required while powered	Required
Latency	Very low	Higher
Bandwidth	Very high for on-chip access	High system bandwidth but higher access latency
Density	Lower	Higher
Cost per bit	Higher	Lower
Power characteristic	Significant static area/leakage cost	Refresh and activation consume energy
Typical use	CPU caches	Main memory
Volatility	Volatile	Volatile

SRAM in Cache Memory

SRAM is well suited for processor caches because its cells can be accessed rapidly and do not need refresh cycles. This allows the processor to retrieve data with very low latency. The

main disadvantage is its relatively large cell area, which makes large SRAM structures expensive in terms of chip area.

Consequently, SRAM is generally used for L1, L2, and L3 caches where low latency is more important than extremely high capacity.

DRAM in Main Memory

DRAM provides much greater storage density than SRAM. This makes it economically suitable for large main-memory systems. However, its cells require periodic refresh and memory accesses involve operations such as row activation, sensing, and precharging.

DRAM therefore has higher access latency than SRAM, although modern DDR technologies provide very high data-transfer rates. For example, Micron's DDR5 documentation lists data rates from 4800 to 8800 MT/s for different DDR5 configurations and identifies higher bandwidth as a major improvement over DDR4.

Latency and Bandwidth Analysis

SRAM has the advantage in **access latency**, making it appropriate for processor caches where every nanosecond or processor cycle can affect execution speed.

DRAM has higher access latency but provides much greater **capacity and cost efficiency**. Its high transfer rates and multiple banks/channels allow it to achieve high sustained bandwidth.

This produces an important architectural trade-off:

SRAM → low latency + high cost + low density

DRAM → higher latency + high density + lower cost per bit

Impact on System Throughput

The processor can execute instructions rapidly when frequently accessed data is available in SRAM cache. If cache misses occur frequently, requests reach DRAM and the processor may experience longer waiting periods.

Therefore, the combination of SRAM cache and DRAM main memory provides a balance between speed and capacity.

Critical Analysis

Using only SRAM for all system memory would provide excellent latency but would require a large amount of expensive chip area. Using only DRAM would provide capacity at lower cost but would significantly increase processor-memory latency.

The most effective design is therefore hierarchical:

CPU → SRAM Cache → DRAM Main Memory

This arrangement improves overall system throughput by keeping frequently accessed data close to the processor.

Conclusion

SRAM is preferred for cache memory because of its low latency and high-speed operation, while DRAM is preferred for main memory because of its high density and economical large-scale capacity. Their complementary characteristics make them fundamental components of hierarchical memory systems.

3. Virtual Memory with Paging vs Cache Memory

Introduction

Virtual memory and cache memory both improve the effective operation of a computer system, but they solve different problems.

Cache memory reduces the average time needed to access frequently used data.

Virtual memory with paging provides a larger logical address space and allows programs to use memory beyond the immediately available physical RAM by transferring pages between storage and main memory.

Feature	Cache Memory	Virtual Memory with Paging
Main objective	Reduce access latency	Provide larger logical memory space
Managed mainly by	Hardware	OS + MMU hardware
Unit	Cache line/block	Page
Typical storage relationship	CPU ↔ cache ↔ DRAM	Virtual address ↔ physical page ↔ storage
Normal access	Very fast	Slower than cache
Miss/failure event	Cache miss	Page fault
Replacement	Cache replacement policies	Page replacement policies
Effect on throughput	Usually improves CPU throughput	Can reduce throughput if page faults are frequent
Main advantage	Fast data access	Memory isolation and expanded address space
Major penalty	Lower-level cache/memory access	Storage access during page fault

Cache Memory Mechanism

When the processor requests data, the cache is checked before accessing lower levels of the memory hierarchy. If the data is present, a **cache hit** occurs. If it is absent, a **cache miss** occurs and the data must be retrieved from a lower memory level.

The purpose is to exploit locality and reduce average memory access time.

Virtual Memory and Paging

Virtual memory divides the virtual address space into fixed-size pages. Physical memory is divided into page frames. A page table maps virtual pages to physical frames.

When a required page is already present in physical memory, the processor can continue after address translation. If the page is not present, a **page fault** occurs. The operating system must retrieve the required page from secondary storage and place it into physical memory.

Access Latency Comparison

Cache access is normally measured in processor cycles and is designed for extremely frequent accesses. Virtual-memory translation itself can be implemented efficiently using hardware structures such as the Translation Lookaside Buffer (TLB), but a page fault is extremely expensive compared with cache access because it can require storage I/O and operating-system intervention.

Therefore:

Cache hit → extremely low latency

Cache miss → access to lower memory hierarchy

Normal paged-memory access → higher overhead due to address translation

Page fault → very large latency penalty

Effect on System Throughput

A high cache-hit rate generally increases CPU throughput because the processor spends less time waiting for memory.

In contrast, frequent page faults can dramatically decrease system throughput because the processor may spend substantial time waiting for storage operations. This is known as **thrashing** when excessive paging prevents useful computation.

Critical Analysis

Cache and virtual memory should not be considered competing technologies. They operate at different levels and solve different problems.

Cache memory answers:

"How can frequently accessed data be obtained faster?"

Virtual memory answers:

"How can programs obtain a protected and flexible logical memory space larger than immediately available physical memory?"

Modern systems combine both mechanisms to achieve performance, protection, flexibility, and efficient memory utilization.

Conclusion

Cache memory primarily improves **access latency and processor throughput**, whereas virtual memory with paging primarily provides **address-space flexibility, protection, and memory management**. Cache misses normally have a manageable penalty, while page faults

can be extremely expensive. Therefore, minimizing both cache misses and unnecessary page faults is important for high-performance systems.

4. NAND Flash vs DRAM

Introduction

NAND Flash and DRAM are fundamentally different memory technologies. DRAM is volatile and is normally used as main memory, whereas NAND Flash is non-volatile and is primarily used for persistent storage such as SSDs, smartphones, memory cards, and embedded storage.

Feature	NAND Flash	DRAM
Volatility	Non-volatile	Volatile
Latency	Much higher than DRAM	Much lower
Bandwidth	High, especially modern NAND interfaces	Very high
Persistence	Retains data without power	Loses data without power
Density	Very high	High
Main use	SSDs and persistent storage	Main memory
Write behavior	Program/erase operations are relatively slow	Fast read/write compared with Flash
Endurance	Limited program/erase cycles	Not based on Flash-style P/E endurance
Hierarchical position	Storage level	Main-memory level

DRAM Characteristics

DRAM provides fast random access and is directly connected to the memory controller in conventional systems. It is therefore positioned much closer to the processor than NAND storage.

Modern DDR5 DRAM supports high data rates. Micron specifies DDR5 data rates ranging from 4800 to 8800 MT/s across its documented configurations, illustrating the high bandwidth available from modern DRAM systems.

NAND Flash Characteristics

NAND Flash is optimized for high density and persistent storage. Modern NAND technologies can provide very high sequential transfer rates. For example, Micron's G9 NAND specifies an I/O transfer rate of up to 3.6 GB/s for its NAND technology.

However, NAND's storage-oriented architecture introduces significantly higher access latency than processor memory. SSD controllers also use techniques such as parallel NAND channels, caching, wear management, and error correction to improve practical performance.

Latency Comparison

DRAM has much lower access latency and is therefore appropriate for active program data.

NAND Flash has higher latency, particularly for program and erase operations. As a result, it is unsuitable as a direct replacement for conventional DRAM in a processor's normal main-memory role.

Bandwidth and Throughput

Modern NAND can provide excellent **sequential throughput**, especially when multiple NAND channels operate in parallel. However, bandwidth alone does not make NAND equivalent to DRAM.

DRAM provides low-latency random access with high sustained bandwidth, making it more suitable for workloads requiring frequent fine-grained memory operations.

NAND is more suitable when **capacity and persistence** are more important than extremely low latency.

Hierarchical Memory Placement

A typical hierarchy can be represented as:

CPU → Cache → DRAM → SSD/NAND Storage

DRAM acts as the main working memory, while NAND provides large-capacity persistent storage.

Critical Analysis

Replacing DRAM completely with NAND would reduce system responsiveness for workloads requiring frequent random accesses. However, NAND is extremely valuable because it provides persistent storage at much higher density than volatile memory.

The two technologies therefore complement each other rather than directly competing.

Conclusion

DRAM is preferred for main memory because of its low latency and high random-access performance. NAND Flash is preferred for persistent storage because of its non-volatility, high density, and large capacity. Modern NAND can provide high throughput, but its latency and access characteristics prevent it from replacing DRAM as conventional main memory in most systems.

5. Comparison of MRAM and RRAM

Introduction

MRAM (Magnetoresistive Random Access Memory) and RRAM (Resistive Random Access Memory), also called ReRAM, are emerging non-volatile memory technologies. Both retain

information without continuous power and are being investigated for applications ranging from embedded memory to storage-class and compute-in-memory architectures.

Feature	MRAM	RRAM
Full form	Magnetoresistive Random Access Memory	Resistive Random Access Memory
Storage principle	Magnetic state/resistance	Resistance state of a switching material
Volatility	Non-volatile	Non-volatile
Read access	Fast	Potentially fast
Write characteristics	Write energy can be significant	Potentially low-energy writes depending on implementation
Endurance	Generally high	Technology-dependent
Density	Good	Potentially very high
Energy efficiency	Good for standby because it is non-volatile	Potentially very good for specific workloads
Main challenge	Write energy/current	Variability, endurance, switching control
Applications	Embedded memory, caches, non-volatile systems	Embedded memory, storage, AI/in-memory computing

MRAM Performance

MRAM stores information using magnetic states rather than electrical charge. Because it is non-volatile, it can retain information when power is removed.

Experimental MRAM research has demonstrated very fast access. For example, an IBM Research/IEEE MRAM test chip reported a predicted random read access time of approximately 5 ns and write pulse widths below 5 ns.

However, write operations can require considerable current and energy. IBM research has specifically studied the energy-delay characteristics of STT-MRAM write operations and identified high write current as an important challenge.

RRAM Performance

RRAM stores information by changing the resistance of a memory cell. Its non-volatile nature allows it to retain information without continuous power.

RRAM is particularly interesting for high-density memory and **in-memory computing**, where computation can be performed close to or inside memory arrays. Recent IBM research describes resistive-memory-based analog in-memory computing as a promising approach for improving computational energy efficiency and reducing data movement.

However, RRAM performance depends strongly on the material, device structure, switching mechanism, and control circuitry. Device-to-device variability and endurance are important engineering considerations.

Access Time

MRAM has demonstrated fast random access and is considered a candidate for low-latency non-volatile memory. RRAM can also provide fast switching, but exact performance varies substantially among implementations.

Therefore, it is not correct to assign one universal latency value to all MRAM or RRAM devices.

Throughput

Throughput depends on array organization, interface width, parallelism, sensing circuits, and workload.

MRAM can support high-speed memory operations, making it attractive for embedded and cache-like applications.

RRAM can achieve substantial effective throughput when many cells operate in parallel. This is particularly valuable for matrix operations and AI accelerators using resistive crossbar arrays.

Energy Efficiency

One major advantage of both technologies is **non-volatility**. Unlike volatile memories, they can retain data without continuous refresh.

MRAM can provide low standby power because information is stored magnetically. However, write operations can consume significant energy.

RRAM can provide attractive energy efficiency for certain workloads, especially in-memory computing, because computation and storage can be brought closer together. However, energy efficiency depends on the specific device and circuit implementation.

Research on hybrid non-volatile caches has also shown that technologies such as MRAM can provide different power-performance characteristics from conventional SRAM, motivating combinations of memory technologies rather than relying on a single technology.

Critical Analysis

MRAM and RRAM should not be viewed simply as direct replacements for each other. MRAM is attractive where fast access, non-volatility, and endurance are important. RRAM is particularly promising for high-density storage and emerging in-memory computing architectures.

The selection depends on the application:

MRAM → fast non-volatile memory and embedded/cache-oriented applications

RRAM → high-density non-volatile memory and compute-in-memory applications

Conclusion

MRAM and RRAM are promising technologies for future hierarchical memory systems. MRAM offers fast access and non-volatility but can face write-energy challenges. RRAM offers high density and strong potential for in-memory computing, although variability and endurance remain important considerations. Both technologies could complement conventional SRAM and DRAM rather than immediately replacing them.

Overall Comparative Summary

Technology	Relative Latency	Bandwidth / Throughput	Volatile?	Main Application
L1 Cache	Extremely low	Extremely high	Yes	Immediate CPU data/instructions
L2 Cache	Very low	Very high	Yes	Backing cache for L1
L3 Cache	Low–moderate	High	Yes	Shared last-level cache
SRAM	Very low	Very high	Yes	CPU cache
DRAM	Moderate	High	Yes	Main memory
Virtual Memory	Higher than cache	Depends on paging behavior	Uses underlying memory/storage	Memory management
NAND Flash	High	High sequential throughput	No	Persistent storage
MRAM	Low	High potential	No	Embedded/non-volatile memory
RRAM	Low–moderate, implementation-dependent	High potential with parallel arrays	No	Dense NVM/in-memory computing

Final Conclusion

The comparison demonstrates that no single memory technology provides the best latency, bandwidth, capacity, cost, energy efficiency, and persistence simultaneously. Modern computer architecture therefore uses a **hierarchical memory system** in which each technology is selected according to its strengths.

L1, L2, and L3 caches reduce processor access latency, with L1 providing the fastest access and L3 providing larger shared capacity. SRAM is suitable for these cache levels because of its speed, whereas DRAM provides the larger capacity required for main memory.

Virtual memory with paging provides address-space flexibility and protection, while cache memory primarily improves access latency. NAND Flash extends the hierarchy into persistent storage, providing high capacity but much higher latency than DRAM.

Emerging technologies such as MRAM and RRAM introduce non-volatility and new possibilities for energy-efficient and in-memory computing systems. Consequently, future multicore systems are likely to continue combining multiple memory technologies rather than depending on a single universal memory.

The overall design principle can therefore be summarized as:

Smaller + faster memory near the processor → larger + slower memory farther from the processor.

This hierarchical approach improves overall processor performance, system throughput, capacity, energy efficiency, and cost effectiveness.

References

1. Intel Corporation, *Memory Performance in a Nutshell*.
2. Intel Corporation, *Improving Real-Time Performance by Utilizing Cache Allocation Technology*.
3. Intel Corporation, *P-Core and E-Core Level 0, Level 1 and Level 2 Caches*.
4. Micron Technology, *DDR5 SDRAM*.
5. Micron Technology, *G9 NAND Flash Memory*.
6. IBM Research, *A High-Speed 128-kb MRAM Core for Future Universal Memory Applications*.
7. IBM Research, *Energy-Delay Optimization of the STT-MRAM Write Operation*.
8. IBM Research, *On-Chip Training and Inference using Analog CMO/HfOx ReRAM Artificial Synapses*.
9. IEEE Technology Navigator, *Cache Memory and Cache Hierarchy*.